TRACK 2 DISPATCH / TEAM LEARNING SERIES

DOCUMENT 1 OF 2

# Understanding Track 2 Dispatch

## The complete beginner's learning guide

A detailed reference to the business problem, every project file, Python concepts, calculation tools, allocation rules, shock monitoring, tests, evidence, and remaining work. Full content of the Markdown learning guide.

**Implementation snapshot: 14 September 2026**
The allocation engine is built. Conversational AI and the dispatcher interface remain planned work.

Prepared for reading, presentation preparation, and team sharing

PDF edition: 15 September 2026

Code lines may wrap in print; use the Markdown source when copying commands.

# Contents

Click a chapter title or use the PDF bookmarks to navigate.

1. The project in everyday language 3

2. What the assignment actually asks for 3

3. A small programming vocabulary 4

4. The folders and every project file 7

5. Understand the data before the code 8

6. Follow one batch through the program 11

7. harness/simulate.py: the practice world 12

8. src/tools.py: checks and arithmetic 13

9. src/objectives.py: what does “best” mean? 15

10. src/allocator.py: choose and remember 17

11. ShockMonitor: notice a queue that behaves unexpectedly 18

12. The heuristic and the shock evaluation 20

13. Tests and measured performance 21

14. What to implement next, and why 23

15. Presentation preparation and deadlines 24

16. Commands to explore the implementation yourself 25

17. Where to edit when a requirement changes 27

Reviewed on 14 September 2026 against the project code and the supplied capstone documents. This is a study guide for the implementation that exists now, not a claim that the final capstone is complete. No simulator or application code was changed for this review.

# 1. The project in everyday language

SweaterCo makes clothing. Sometimes its own factory has too much work, so it sends a batch of garments to an outside workshop. There are eight workshops. They differ in speed, current workload, price, quality, and the kinds of garments they can make. Some are not allowed to receive certain jobs.

A dispatcher is the person who decides where each batch should go. Our intended product helps that person turn a message such as “Send these 800 scarves somewhere that can meet Friday” into an informed, explainable allocation.

The completed product should understand the message, ask about missing information, check the available facts, compare workshops, explain the trade-offs, and record the decision. It should not invent an order, promise an impossible date, or send work to a suspended workshop.

**What exists now:** the Python calculation tools, configurable scoring rules, an allocator that chooses a workshop, records explaining those choices, a queue-disruption monitor, a small comparison policy, a shock evaluation script, and automated tests.

**What does not exist yet:** a working conversational AI that processes the 30 messages, a dispatcher interface, a complete persistent decision history, implemented order splitting, and the final evaluation/submission package.

The current allocation engine does not call an LLM. An LLM is a large language model: software that interprets and generates language. Using ChatGPT to help write Python does not mean the resulting Python program itself calls ChatGPT. Our current engine follows explicit rules and arithmetic.

# 2. What the assignment actually asks for

I reviewed all seven PDFs in `Documents_Capstone`: the original, v3, and v4 project specifications; the original, v2, and v3 track descriptions; and the sprint-review example. I also reviewed the project's factory primer in Markdown and PDF. The latest supplied specification is `Capstone_Project_Spec_v4.pdf`, and it refers to `Capstone_Project_Tracks_v3.pdf`.

The older versions are useful background, but some instructions changed. In particular, the team chooses its own primary objective and declares it at Sprint 1. It is not an objective assigned to the team at formation. The latest naming rule uses a two-digit group number, such as `Group01`. Track 4's prohibition on Canvas/LMS integration is about Track 4, not a feature we must implement for Track 2.

Our track is **The Quick-Response Subcontracting Desk**. Track 1 is the general manager's broader factory assistant. The other tracks concern research writing, student productivity, or an approved custom proposal. Their special evaluation requirements should not be substituted for ours.

| Track 2 requirement | Meaning | Current position |
|---|---|---|
| Natural-language intake | Handle all 30 supplied chat requests, including ambiguity and unavailable facts | Not implemented |
| Capability, capacity, eligibility tools | Check what each workshop can make, its workload, and its restrictions | Implemented for structured inputs |
| Estimates before selection | Calculate time and cost from data | Implemented, with documented model limits |
| Configurable objectives | Declare a primary objective; also evaluate another objective and a weighted hybrid | Six presets and file loading implemented; the team still needs to declare and justify its choice |
| Explainable allocation | Keep alternatives, estimates, reasons, and constraints | DecisionRecord implemented; persistent history and constraint author/source still missing |
| Awkward cases | Acknowledge when no legal or on-time choice exists | Structured fallback records implemented; conversational handling missing |
| Disruption response | Evaluate a hidden workshop closure | Monitor and trace implemented |
| Usable interface | A dispatcher can understand the recommendation and alternatives at a glance | Not implemented |
| Honest evaluation | Compare all three supplied baselines, a strong short heuristic, other objectives, a hybrid, and shock runs | Tests and shock trace exist; this review also ran a broader comparison; final report still missing |
| Chat evaluation | Score all 30 requests by extract/clarify/refuse/decline; judge decisions and explanations separately | Not implemented |
| Failure analysis | Analyse at least ten individual failures and explain their causes | Not yet a completed evaluation artifact |

The track description explicitly says that an allocation policy without a natural-language surface does not meet the core capabilities. Good simulator results are therefore necessary evidence, but they are not the whole project.

The intended division of work is straightforward: language interpretation belongs in the future AI layer; eligibility, dates, costs, queues, and scoring belong in code tools. The model should explain tool results rather than guess numbers.

According to the supplied v4 specification, the project is 40% of the course grade. Within the project, problem solving is 35%, evaluation 15%, interface 15%, engineering 20%, documentation 8%, and presentation 7%. This gives us a practical reason to finish the interface and evaluation rather than spend all our time changing the allocation formula.

## 3. A small programming vocabulary

You do not need to memorise every symbol before understanding the business logic. Use this table when reading a line of code.

| Python term or notation | Plain-English meaning | Example in this project |
|---|---|---|
| Variable | A name holding a value | `processing_days` holds how long the work takes |
| `str` | Text | `"W2"` or `"TOPS"` |
| `int` | A whole number | 340 pieces |
| `float` | A number that can have a decimal part | 2.5 queue days |
| `bool` | A yes/no value | `True` means eligible |
| `None` | No value or no choice | No maximum cap, or no workshop selected |
| Dictionary, `dict` | A record with named fields | `{"pieces": 340, "category": "TOPS"}` |
| List | A collection in an order | The candidate workshop rows |
| Set | A collection of unique members, useful for membership checks | Categories a workshop can make |
| `def` / function | A reusable recipe with inputs and an answer | `estimate_cost(...)` |
| Parameter / argument | An input named in the recipe / the value supplied when using it | `workshop_id` / `"W2"` |
| `return` | Give the answer back to the caller | Return a dictionary of estimates |
| `class` | A definition of a kind of object | `Allocator` |
| Object / instance | One working copy of that definition | One allocator with its own run history |
| `self` | This particular object's own data | `self.pieces_by_workshop` |
| `__init__` | Setup performed when an object is created | Start with empty allocation history |
| `__call__` | Let an object be used like a function | `allocator(batch, workshops, queues)` |
| Dataclass | A convenient definition of a structured form | `Workshop`, `ObjectiveConfig`, `DecisionRecord` |
| `import` | Make another module's functions or classes available | Import `Simulator` from `simulate` |
| Module | A Python file | `tools.py` is the module `tools` |
| `if`, `elif`, `else` | Test conditions and select a branch | Suspended, wrong category, too large, otherwise okay |
| `for` | Repeat a step for each member | Check all eight workshops |
| `continue` | Skip to the next member of a loop | Skip an ineligible workshop |
| `raise ValueError(...)` | Stop and explicitly report an invalid situation | No legal workshop for a simulator dispatch |
| `lambda` | A small unnamed function, often a ranking rule | Rank by total completion time |

| Python term or notation | Plain-English meaning | Example in this project |
|---|---|---|
| `min`, `max`, `sum`, `sorted` | Smallest, largest, total, ordered collection | Pick the lowest score |
| `.get(key, default)` | Look up a value, with a fallback when absent | Previously allocated pieces default to zero |
| `.values()` / `.items()` | Read the values / key-value pairs in a dictionary | Visit all workshop objects |
| `**some_dict` in a dictionary | Include the fields from another dictionary | Merge ETA and cost into a candidate |
| List comprehension | A compact loop that builds a list | Keep only candidates that meet the date |
| `f"...{value}..."` | Insert a value into a text template | Put the computed score into a reason |
| Type annotation, such as `-> float` | A hint about the expected type | The score should be a decimal number |
| `TYPE_CHECKING` | Imports needed by development tools rather than normal execution | Refer to `Workshop` without a runtime import |
| `from __future__ import annotations` | Defer interpreting type annotations | Support these type references conveniently |
| `Path(__file__)` | Locate this Python file on disk | Find the nearby data folder |
| `with open(...)` | Open a file and close it when finished | Read a CSV |
| `if __name__ == "__main__"` | Run this section when the file is launched directly | Start the simulator's command-line program |

Type annotations alone do not validate user input. A field labelled `int` is not a complete safeguard against a malformed chat request. We still need explicit validation in the future intake layer.

Indentation matters in Python: indented lines belong to the preceding function, loop, or condition. Comments beginning with `#` and descriptive triple-quoted docstrings explain the code; they are not allocation decisions.

# 4. The folders and every project file

| File or directory | Job |
|---|---|
| `data/` | Input information: orders, workshop profiles, and chat requests |
| `harness/` | The supplied simulation environment used to evaluate policies |
| `src/` | Our reusable application logic |
| `eval/` | Evaluation code and saved evidence |
| `docs/` | Explanations of the system and its tools |
| `tests/` | Automated checks that verify expected behaviour |
| `README.md` | Initial setup instructions and the project layout |
| `requirements.txt` | Names of external Python packages to install |
| `.env.example` | An empty example `OPENAI_API_KEY=` setting |
| `.gitignore` | Rules telling Git which local/generated files to leave out |
| `Factory_Primer.md` | The manufacturing story, process, and glossary |
| `pdfs/Factory_Primer.pdf` | PDF version of that background |
| `data/data_dictionary.md` | Explanation of each data file and column |
| `data/orders.csv` | The 120 customer orders |
| `data/workshops.csv` | The eight workshop profile cards |
| `data/dispatch_requests.txt` | The 30 messages the future conversational system must handle |
| `harness/simulate.py` | Loads data, replays orders, simulates outcomes, and reports metrics |
| `src/tools.py` | Six calculation/checking functions and the stateful ShockMonitor |
| `src/objectives.py` | Objective settings, file loading, score components, and weighted scoring |
| `src/allocator.py` | Selection procedure, run memory, and DecisionRecord |
| `src/heuristic.py` | The short earliest-finish comparison policy |
| `eval/shock_trace.py` | Compares random dispatch and our allocator during a hidden closure |
| `eval/shock_trace.csv` | Saved daily piece counts and flag indicators from that comparison |
| `eval/shock_events.json` | Saved detection/recovery evidence for the default trace |
| `docs/tool_spec.md` | Input, output, purpose, non-goals, failure modes, and arithmetic caveats |
| `tests/conftest.py` | Makes `src/` and `harness/` importable during tests |
| `tests/test_tools.py` | Tests for the six tools |
| `tests/test_allocator.py` | Tests for objectives, decisions, simulation integration, and the heuristic |
| `tests/test_shock_monitor.py` | Tests for detection, recovery, and the trace program |

This guide is an additional documentation file created during the review. The existing project had 23 tracked files before it was added.

`.venv` is a private toolbox containing this project's Python environment and installed packages. It is not our application source. `.git` holds version history. `__pycache__` and `.pytest_cache` are generated caches. We do not need to study their internals to understand the application.

The dependencies currently listed are `pandas`, `streamlit`, `python-dotenv`, `pytest`, `openai`, `matplotlib`, and `PyYAML`. They respectively support table analysis, a web interface, environment settings, tests, model API access, charts, and YAML reading. **Being listed does not mean a feature exists.** The current core mostly uses Python's built-in modules. PyYAML supports the config loader and pytest runs tests; no Streamlit interface or OpenAI call is implemented yet.

`.env.example` is safe to share because its value is empty. A real `.env` is local and ignored. Copying it does not automatically load it into Python. No API key is required for the simulator or current tests.

Git is the version-history system; GitHub is where a copy can be shared. The standalone repository is inside `Track2_dispatch` and points to `apurwakumari16nus/Track2_Dispatcher`. Publishing source code to GitHub is different from hosting a working web application.

# 5. Understand the data before the code

## Orders

`orders.csv` contains 120 orders totalling 93,500 pieces: 85 TOPS orders and 35 ACCESSORIES orders. In the recorded dataset, 86 are COMPLETE and 34 IN_PROGRESS.

`order_id` is the unique reference. `customer` identifies the buyer, `product` describes the item, `category` is the broader capability category, and `pieces` is the quantity. `order_date` and `due_date` describe the schedule. `status`, `current_stage`, and `last_activity_date` describe the recorded progress. `completed_date` and `days_late` describe historical completion; they are blank for unfinished jobs. A negative historical `days_late` means the job finished early.

The simulator does not train on these historical outcomes. It replays all 120 orders using only order ID, category, quantity, order date as dispatch date, and due date. It does not skip an order because the CSV currently says COMPLETE. It simulates a new allocation experiment.

## Workshop profiles

| ID and name | Pieces / day | Lead days | Batch defect chance | Cost / piece | Makes | Restriction | Initial queue days |
|---|---|---|---|---|---|---|---|
| W1 QuickStitch | 300 | 1 | 12% | 1.50 | TOPS | Active, no batch cap | 4 |
| W2 SteadyHands | 170 | 2 | 3% | 1.30 | Both | Active, no batch cap | 2.5 |
| W3 BudgetWorks | 230 | 3 | 9% | 0.80 | TOPS | Active, no batch cap | 5 |
| W4 Little Loom | 70 | 1 | 2% | 1.60 | ACCESSORIES | Active, no batch cap | 0.5 |
| W5 GiantWeave | 400 | 4 | 6% | 1.10 | TOPS | Active, no batch cap | 3 |
| W6 Nimble Needle | 130 | 1 | 5% | 1.20 | Both | Active, no batch cap | 1 |
| W7 OldMill | 200 | 2 | 10% | 0.90 | Both | Suspended | 0 |
| W8 FreshStart | 100 | 2 | 4% | 1.00 | TOPS | Active, maximum 300 pieces/batch | 0 |

“Both” means TOPS and ACCESSORIES. Cost is in the simulator's price units; the data does not establish a currency to label these amounts with.

Capacity is a processing speed, not necessarily a maximum order size. A workshop making 170 pieces per day can accept 340 pieces and take two days. A separate maximum-batch cap is a legal restriction: FreshStart must reject 301 pieces even if there is plenty of time.

Queue days describe work ahead of the new batch. Lead days represent transport overhead. Transport delays delivery but does not keep the workshop's production line occupied after the work is made.

The defect rate means the chance that a **batch** requires rework. It does not mean the simulator marks that percentage of individual garments defective. When a defect occurs, the simulator adds 50% to that batch's processing time.

The initial CSV queue is used at the start of a simulation. Later decisions must use the live `queues` argument, because allocations and elapsed time have changed the workload. The descriptive `notes` column is not used by the current scoring formula.

## Two different clocks

The chat dataset says “today” is **1 April 2026**. The simulator instead replays historical dispatch dates, beginning **1 January 2026** and ending **27 March 2026**, its day 85. Shock day 33 is therefore 3 February, not 4 May.

Some chat messages have deadlines already in the past on 1 April. The future assistant must notice this. It must not quietly pretend that today is the original order date.

The primer describes a Monday-to-Saturday factory and four stages: knitting, assembly, washing, packing. The shared simulator uses calendar-day arithmetic and a single processing duration. It does not simulate

those four stages individually or skip Sundays. We must describe the implemented model accurately and preserve the supplied harness.

## The 30 chat requests

The inbox deliberately includes messy language. The following is a reading guide, not a completed set of measured agent answers. Several requests require more than one behaviour, such as extracting the order and warning about a past deadline.

| Request | Important issue for the future assistant |
|---|---|
| R01 | “Usual quantities” is not a reliable explicit quantity; clarify the job/details |
| R02 | Previous October's chosen workshop is not available as allocation history in these files |
| R03 | A customer reference alone does not uniquely identify the order |
| R04 | ORD-029, 1,500 beanies: the April 1 deadline is today |
| R05 | The deadline has changed but the new date is missing; ask rather than reuse the old date |
| R06 | ORD-061, 800 scarves, April 5: extract and compare workshops |
| R07 | ORD-020, 1,200 vests, March 31: the requested date is already past |
| R08 | ORD-073 is 400 hoodies; FreshStart's 300-piece cap rules it out |
| R09 | ORD-045, 150 vests: an urgent speed preference |
| R10 | A new rush request is missing essential details |
| R11 | OldMill is suspended; explain refusal and offer eligible alternatives |
| R12 | ORD-053: exclude QuickStitch and preserve that the boss imposed the constraint |
| R13 | A customer selling-price quotation is not present; workshop production cost is not that quotation |
| R14 | QuickStitch cannot make scarves because they are ACCESSORIES |
| R15 | A customer-only reference is ambiguous |
| R16 | ORD-114, 2,000 hoodies: its March 23 due date is past |
| R17 | ORD-120, 1,500 vests: its March 17 due date is past |
| R18 | ORD-999 does not exist in the provided order file |
| R19 | “Big reorder” is not enough to identify a batch |
| R20 | ORD-103, 2,000 scarves, April 4: a large batch against a near deadline |
| R21 | ORD-081: seek low cost subject to meeting April 13 |
| R22 | ORD-083, 1,000 hoodies: the March 23 date is past |
| R23 | ORD-093: splitting is suggested; its March 29 date is also past |
| R24 | Little Loom cannot make ORD-015's TOPS category |

| Request | Important issue for the future assistant |
|---|---|
| R25 | ORD-109: exclude BudgetWorks |
| R26 | ORD-107, 2,000 cardigans: the March 16 date is past |
| R27 | ORD-055: prefer low defects, but March 31 is already past |
| R28 | ORD-002: March 29 is already past |
| R29 | A reference to “vests” is ambiguous |
| R30 | ORD-099, 1,500 polos: compare against April 5 |

**Extract** means turn a sufficiently clear message into structured fields. **Clarify** means ask for missing or ambiguous information. **Refuse** means explain why a proposed illegal workshop cannot take the job and offer an alternative where possible. **Decline to answer** means acknowledge that the requested fact is unavailable, such as an unknown historical allocation or selling price. These are different from simply returning the fastest workshop.

# 6. Follow one batch through the program

The simulator gives the allocator three inputs:

```
batch = {
    "order_id": "EXAMPLE-340",
    "category": "TOPS",
    "pieces": 340,
    "sent_date": "2026-04-01",
    "due_date": "2026-04-10",
}
```

The real simulator uses Python date objects; our tools also accept these ISO date strings. `workshops` is a dictionary of Workshop objects keyed by W1 through W8. `queues` is another dictionary, such as `{"W2": 2.5, ...}`, containing current waiting time at each workshop.

The current path is:

```
CSV order and workshop data
          |
          v
Simulator supplies batch, profiles, current queues
          |
          v
Allocator checks observed queue surprises
          |
          v
Tools remove illegal/excluded shops and estimate each remaining option
          |
          v
Objectives assign numerical penalties
          |
          v
Allocator applies date/fallback rules and chooses a workshop
          |
          +--> DecisionRecord explains the choice and saves current evidence
          +--> Internal piece history and expected queue are updated
          |
          v
Simulator checks legality and simulates actual work, rework, and closure delays
          |
          v
Next batch arrives with updated observed queues
```

The simulator is the practice world; the allocator is the decision-maker being tested. They do different jobs. The allocator cannot inspect the simulator's future random outcomes.

## 7. `harness/simulate.py`: the practice world

The main parts are `Workshop` at line 66, `eligible_workshops` at line 80, `load_workshops` at line 95, `load_batches` at line 110, `Simulator` at line 127, the three baselines near line 209, and the command-line entry at line 227.

`Workshop` is a profile form. The loader translates longer CSV names into shorter attributes: `capacity_pieces_per_day` becomes `capacity`, `pickup_lead_days` becomes `lead_days`, `cost_per_piece` becomes `cost`, and `current_queue_days` becomes `queue0`. It turns numeric text into numbers, the makes field into a set of categories, and a blank maximum cap into `None`.

`eligible_workshops` is the official rule. It rejects non-ACTIVE profiles, unsupported categories, and batches above a cap. It returns Workshop objects. It does not reject a workshop merely because its queue is long; a legal but slow allocation is allowed and may score badly.

`load_batches` reads all orders, selects the five fields the allocator needs, converts dates/quantities, and sorts by dispatch date. The data path is calculated from the simulator file's own location, so `harness/../data/` works without depending on the terminal's current directory.

`Simulator.__init__` loads profiles and batches and remembers whether to introduce a shock and which random seed to use. A seed is a starting setting that makes the sequence of simulated random events reproducible.

`Simulator.run` starts fresh queues for each policy run. Internally, it stores the day number when each workshop finishes its existing work. Before a dispatch, it converts that number into remaining queue days: `max(0, free_day - current_day)`. A queue cannot become negative just because the workshop has been idle.

It asks the policy to return a workshop ID, then checks that ID against the official eligible set. An illegal answer raises an error immediately. After a legal answer, it computes when production can start, applies any relevant hidden closure, calculates work time, samples whether rework occurs, updates the queue, adds transport time, and records the result.

The normal processing rule is `pieces / capacity`. A sampled defect multiplies that processing time by 1.5. Cost is always `pieces * cost_per_piece` in this model; it does not separately charge for rework or tax.

Under `--shock`, the simulator picks one active workshop with its seeded random generator. With seed 5105 it picks W2. If that workshop's planned production start falls between day 30 and day 44 inclusive, the code moves that start to day 44. A start already at 44 gains no extra delay, so the trace labels days 30-43 as the closure window. The condition concerns **production start**, not just the day on which a batch was dispatched.

`report` measures average turnaround, a 90th-percentile turnaround, the percentage of batches late, the percentage of batches defective, total model cost, and the largest workshop's share of allocated pieces. Mean and defect/late rates count batches, not pieces. Maximum share counts pieces. The p90 implementation selects index `int(0.9 * n)` from the sorted durations; it is not an interpolated spreadsheet percentile.

Running the file directly runs only `random`, `greedy_biggest`, and `cheapest`. It does not automatically import and run our Allocator. The random baseline makes repeatable order-based choices, the biggest baseline picks highest capacity without considering the queue or transport, and the cheapest picks the lowest unit price without considering waiting time.

The harness remains unchanged. Its simplifications define the shared comparison, even when a richer real-world model would be attractive.

## 8. `src/tools.py`: checks and arithmetic

The six public functions are pure: they do not contact a model, access the network, write files, change the supplied inputs, or remember previous calls. The same inputs produce the same results. A function can still raise an error when inputs are invalid. Their dictionary/list outputs are convenient to inspect and convert to JSON.

`_as_date` at line 19 is an internal helper. It converts a `YYYY-MM-DD` string to a date and leaves a date object as a date.

| Function | What it does, step by step | What it does not do |
|---|---|---|
| `check_eligibility`, line 24 | Visit every workshop; check ACTIVE status, category, then cap; return ID, name, eligible boolean, and first failing reason | Does not rank by speed or check live disruption |
| `queue_check`, line 47 | Look up current queue, capacity, and lead days for one ID | Does not substitute the initial CSV queue when a live queue is missing |
| `estimate_eta`, line 62 | Divide quantity by capacity; add queue and lead; round days for a calendar ETA; compare ETA with due date; report defect probability | Does not predict the sampled defect or future hidden closure; does not itself reject illegal shops |
| `estimate_cost`, line 94 | Return unit price and quantity multiplied by price | Does not add fees absent from the simulator or quote a customer's selling price |
| `overload_probe`, line 106 | Calculate queue before, queue after adding nominal processing, and added delay for immediately following work | Does not actually modify the queue; lead time is not added to production workload |
| `candidates`, line 128 | Use eligibility; remove requested exclusions; merge ETA and cost; sort by total days | Does not select a winner, apply objective weights, or interpret a whole sentence |

The exact eligibility reasons are `ok`, `suspended`, `cannot make <category>`, and `batch of N exceeds M-piece cap`. The order matters: OldMill is reported suspended first, even if another rule would also fail.

Exclusions match complete IDs or complete names, ignoring case and spaces at the ends. `" w2 "` and `"SteadyHands"` both exclude W2. A partial or misspelled name does not match and is silently ineffective; the future chat layer should resolve or clarify names first.

The four central calculations are:

```
processing_days = pieces / capacity_per_day
total_days      = queue_days + processing_days + lead_days
total_cost      = pieces * cost_per_piece
queue_after     = queue_days + processing_days
```

The tool's date rule is `today + round(total_days).days_to_spare` is due date minus ETA: 3 means three days early, 0 means on the date, and -3 means three days late.

These are normal-condition estimates, not guarantees. A missing workshop/queue produces `KeyError`; capacity zero produces `ZeroDivisionError`; a malformed ISO date produces `ValueError`. Negative quantities and other malformed numeric data are not comprehensively validated here because the functions assume the clean simulator schema. That is another job for the future intake validation.

There is a documented rounding edge case. Python rounds exact halves to the nearest even integer: 2.5 becomes 2, 3.5 becomes 4. The tool rounds the duration from today, while the harness rounds an absolute completion day counted from the run's first day. On an odd elapsed day with an exact half-day duration, dates can differ by one day. For example, day zero April 1, today April 2, and duration 2.5 produce April 4 in the tool and April 5 in the harness. This is not a hidden improvement we should claim: it is a remaining consistency issue to resolve in a future adapter while preserving the shared simulator.

## A complete hand calculation

Use the illustrative 340-piece TOPS batch above, initial queues, today April 1, and due date April 10. This is a teaching example, not an extra row added to `orders.csv`.

For W2 SteadyHands, production takes `340 / 170 = 2` days. Add its 2.5-day queue and 2-day lead: total 6.5 days. Python's `round(6.5)` is 6, so ETA is April 7 and spare time is three days. Cost is `340 * 1.30 = 442`. Defect probability is 3%. Its nominal queue after allocation is `2.5 + 2 = 4.5` days, not 6.5 days, because transport does not occupy production.

The complete eligible comparison is:

| Workshop | Total days | ETA | Days to spare | Model cost | Batch defect chance |
|---|---|---|---|---|---|
| W6 Nimble Needle | 4.6154 | April 6 | 4 | 408 | 5% |
| W1 QuickStitch | 6.1333 | April 7 | 3 | 510 | 12% |
| W2 SteadyHands | 6.5000 | April 7 | 3 | 442 | 3% |
| W5 GiantWeave | 7.8500 | April 9 | 1 | 374 | 6% |
| W3 BudgetWorks | 9.4783 | April 10 | 0 | 272 | 9% |

W4 cannot make TOPS. W7 is suspended. W8 cannot take 340 pieces because its cap is 300. The lateness preset chooses W6 here, the defect preset chooses W2, and a custom cost-only setting chooses W3. That is why “best” needs a stated business objective.

Python may internally represent a multiplication such as `340 * 1.1` as `374.00000000000006`. That is normal binary floating-point representation. We format it for display; tests use approximate comparisons where appropriate. The calculations retain unrounded intermediate values.

## 9. `src/objectives.py`: what does “best” mean?

An objective is a preference used to rank legal choices. It is different from a hard rule. Suspension and product capability are hard rules; lower cost or lower defect probability are preferences unless an additional explicit constraint makes them mandatory.

`ObjectiveConfig`, line 22, is a settings form with five weights: lateness, defect, fairness, robustness, and cost. A weight of zero ignores that term. Larger weights make that term more influential. There is also a name used in decision records. The dataclass is frozen, which discourages accidental changes to shared presets.

`__post_init__` validates the completed form: the name must be nonempty; each weight must be numeric, finite, nonnegative, and not a boolean. Missing weights default to zero. The code does not require the weights to sum to one and does not automatically rescale them.

`PRESETS`, line 49, supplies six ready-made forms:

| Name | Nonzero weights |
|---|---|
| `lateness` | Lateness 1 |
| `defect` | Defect 1 |
| `fairness` | Fairness 1 |
| `robustness` | Robustness 1 |
| `hybrid_lateness_fairness` | Lateness 0.7, fairness 0.3 |
| `hybrid_lateness_defect` | Lateness 0.6, defect 0.4 |

Cost is supported but has no named preset in this dictionary. A custom `ObjectiveConfig(w_cost=1, name="cost")` can use it.

`load_config`, line 63, reads a flat JSON, YAML, or YML file. For example, a file could contain `{"name": "my_hybrid", "w_lateness": 0.7, "w_fairness": 0.3}`. The loader rejects unknown fields, unsupported file extensions, or a structure that is not a mapping. YAML uses `safe_load`. If no name is supplied, the filename without its extension supplies the name. The loading capability exists; there is not yet a committed collection of objective configuration files.

`score_terms`, line 89, calculates five separate unweighted penalties so they can be inspected. `score_candidate`, line 108, multiplies each penalty by its weight and adds the results. **Lower is better.** This is an explicit formula, not a trained model or a mysterious AI confidence score.

Let `late = max(0, -days_to_spare)`, `t = total_days`, and `share` be the workshop's pieces so far divided by all pieces allocated so far. At the start of a run share is zero for every workshop. Let `flag` be 1 if the monitor currently suspects this workshop and 0 otherwise.

```
Lateness   L = late / (late + 30) + 0.01 * t / (t + 30)
Defect     D = expected_defect_rate
Fairness   F = share
Robustness R = (flag + 0.25 * share) / 1.25
Cost       C = cost_per_piece / highest_cost_per_piece_in_all_workshops

score = w_lateness*L + w_defect*D + w_fairness*F
        + w_robustness*R + w_cost*C
```

The division brings unlike quantities onto roughly comparable scales. Otherwise a number such as 30 late days would overwhelm a defect probability such as 0.03 simply because of its unit. Lateness stays below 1.01; the other terms are between zero and one for valid inputs.

For lateness, an on-time candidate has no late-day penalty but retains a small speed preference. Thirty late days contribute `30/(30+30) = 0.5` before the speed addition. The number 30 is a design scale, not a deadline promised to customers. The small speed term resolves equal-lateness comparisons but is not a separately enforced lexicographic rule for every conceivable extreme input.

For fairness, a shop that has received 2,000 of the run's 10,000 allocated pieces gets 0.2. Penalising that share encourages distribution. This is fairness of business volume across workshops, not demographic fairness, equal profits, or allocation proportional to capacity. Shares describe history **before** this batch is added. Large indivisible batches and eligibility restrictions can prevent equal allocation.

For robustness, being flagged contributes a large penalty and existing concentration adds a smaller one. With no flags, `R = 0.2 * share`, so the pure robustness preset ranks like fairness until anomalies appear. A flag is a soft scoring input; it does not make a workshop legally ineligible.

For cost, the price reference uses all workshop profiles, even currently ineligible ones. In this dataset the maximum unit price is 1.60. This keeps the reference from changing merely because a filter removes a workshop. With no positive maximum the term returns zero.

The hybrid weights express preferences over these normalised penalties. A 0.7 weight does not mean 70% of jobs will be on time or that 70% of a business benefit has been achieved. The chosen weights and constants need business justification and sensitivity analysis. They have not been learned automatically.

## 10. `src/allocator.py`: choose and remember

`Allocator` at line 45 is the coordinator. It connects the tools and scoring rules. It holds a configuration, `pieces_by_workshop`, a ShockMonitor, the first decision date, and `last_decision`.

Create a fresh allocator for each independent simulation. Reusing an old one would carry its piece shares and shock expectations into the next experiment. This memory is ordinary Python data stored while the program runs; it is not model training and does not survive a restart automatically.

`__call__`, line 61, is the adapter the simulator uses. It supplies the batch's sent date to `decide`, then returns only the chosen ID. If there is no legal choice, it raises `ValueError` because the simulator requires a real legal workshop. The fuller record remains available in `last_decision`.

`decide`, line 72, follows this sequence:

1. Interpret today, set the first-decision date if necessary, and calculate the elapsed day.
2. Ask the monitor to compare current queues with predictions.
3. Build eligible candidates after user exclusions.
4. Attach each candidate's shock flag, five score components, and total score.
5. Sort by score, then total duration, then workshop ID for deterministic ties.
6. Start a DecisionRecord with alternatives, profile-based rejections, constraints, and shock evidence.
7. Handle an empty candidate list or an impossible date explicitly.
8. Choose a candidate and build an explanation from its computed fields.
9. Predict the selected workshop's new queue, add the batch's pieces to history, and save `last_decision`.

`must_make_date=True` means choose among on-time candidates if any exist. `False`, the default used through the simulator adapter, lets the objective choose a late candidate even when an on-time one exists. A pure defect objective can therefore prefer quality over the date.

If every remaining candidate is late, the allocator records `awkward_case="none_makes_date"` and chooses the least late by rounded spare days, then duration, then score and ID. This fallback applies regardless of the configured objective. It records options to accept lateness or consider a split. The split is only a suggestion; there is no splitting algorithm yet. A true date constraint is recorded as unsatisfied rather than magically made achievable.

If no candidates remain, `awkward_case="no_eligible"`, the choice stays `None`, and the option is escalation for human review. Excluding every workshop can cause this too. Excluded workshops are represented in the

constraints, while `ineligible` contains failures of the workshop-profile rules; those are different reasons for removal.

`decide()` is **not a harmless preview**. A successful call updates allocation memory. Calling it twice for the same batch counts the pieces twice across the two calls. “Exactly once” means once within each successful call, not deduplication across repeated order IDs. Before building a confirm button or repeated chat previews, we should separate preview from commitment and prevent accidental duplicate decisions.

### The DecisionRecord form

`DecisionRecord`, line 19, is the receipt for a decision. `to_dict`, line 41, converts the dataclass and its contents into plain Python dictionaries/lists using `asdict`. `json.dumps(record.to_dict())` converts that into JSON text suitable for saving or sending to an interface.

| Field | What a human can learn from it |
|---|---|
| `order_id` | Which job was considered |
| `chosen_workshop` | Selected ID, or None when no legal selection exists |
| `objective_name` | Which named preference setting was used |
| `candidates` | Every eligible, non-excluded alternative, including time, cost, flag, score components, and score |
| `ineligible` | Workshop-profile rejections with reasons |
| `reason_chosen` | A fixed sentence populated from the selected candidate's computed numbers |
| `constraints_applied` | Exclusions, whether the date was required, and whether the chosen ETA meets it |
| `awkward_case` | No eligible option, no on-time option, or no awkward case |
| `timestamp` | Supplied decision date at midnight, not actual computer wall-clock time |
| `options` | Escalation or late/split suggestions when needed |
| `shock_flags` | Copies of currently active suspect evidence |
| `shock_events` | New detections and recoveries at this decision |

The template cannot independently invent a reason about personal reputation or a phone call: it contains values such as score, total days, spare days, defect probability, and unit price. That improves traceability. It is not yet a friendly conversational explanation of all business trade-offs.

JSON-ready does not mean automatically saved. The allocator retains the last record in memory. The trace collects records while running, then saves selected shock evidence. A complete persistent audit log still needs to be built, including original message, exact objective weights, constraint source/author, input snapshot, and real event timestamp.

## 11. ShockMonitor: notice a queue that behaves unexpectedly

ShockMonitor is in `tools.py` at line 159. Unlike the six pure functions, it is intentionally stateful: it remembers prior predictions and flags. It never receives the hidden workshop ID, the closure dates, or the seed.

Imagine expecting a workshop to have five days of work left on Monday. On Tuesday, with no new allocation there, you expect four. If it now reports twelve, there are eight unexplained extra days. Something has gone wrong, even if you do not know the cause.

| Method or field | Meaning |
|---|---|
| `__init__(threshold=3.0)` | Set the allowable surprise and create empty memory |
| `expected_queue` | Nominal workload predicted just after our most recent allocation to each visited workshop |
| `_expected_on_day` | The day each forecast was made |
| `_last_day` / `_check_day` | Ensure observations do not move backwards in time |
| `_excess` | The most recently measured unexplained extra workload |
| `flags` | Current suspects and their original detection evidence |
| `cleared_flags` | Recovery events from the current observation call |
| `observe(queues, day)`, line 190 | Age predictions, compare with observed queues, raise new flags, and clear recovered ones |
| `record_allocation`, line 229 | Store the nominal queue forecast after a new allocation and its day |
| `is_flagged`, line 242 | Answer whether a workshop is currently suspect |
| `clear`, line 245 | Remove a flag without erasing allocation history |

The aged expectation is `max(0, previous_forecast - days_elapsed)`. Subtracting elapsed time is necessary; a workshop should be working through its queue. Multiple decisions on the same day do not consume that expected work multiple times.

The jump is `observed - expected`. It must be **strictly greater than three days** to trigger the default threshold. Exactly three does not trigger it. A workshop to which we have never sent work has no forecast and cannot be diagnosed by this monitor yet.

A new event records workshop ID, expected queue, observed queue, size of jump, and day flagged. Repeated observations while the same flag remains active preserve the original evidence rather than creating a new first-detection event each time. When the excess falls back to at most the threshold, the monitor clears the flag and records recovery. Missing observations leave flags alone.

After allocation, the allocator obtains `queue_after` from `overload_probe` and gives it to `record_allocation`. If the shop is already suspect, the monitor subtracts the measured unexplained excess before storing the new forecast. Otherwise, sending more work could accidentally redefine an unexplained delay as normal and erase the useful warning.

A flag means **queue anomaly**, not proven closure. Rework can also cause a jump. Small delays can be missed. Detection requires subsequent observations. Other objectives, an all-late fallback, or having only one legal workshop can still send work to a flagged shop.

# 12. The heuristic and the shock evaluation

`src/heuristic.py` contains `earliest_finish`: get the officially eligible workshops and choose the smallest `queue + pieces/capacity + lead`. It raises an error if none exist. Exact ties follow workshop input order. It has no memory, model call, objective settings, or audit receipt.

This is an important comparison because complicated code is not automatically better decision-making. Our richer allocator adds configuration, explanations, constraints, and anomaly evidence, but must still be compared honestly on outcomes.

`eval/shock_trace.py` always runs a shock simulation. `run_trace`, line 44, runs random dispatch and a fresh Allocator independently on the same supplied data/seed. Its default objective is robustness and default seed is 5105. The evaluator reproduces the initial seeded target selection to label the result. That information stays in the evaluator and is not given to the allocator.

The script aggregates target-workshop piece counts for every calendar day, including zero-dispatch days. It tracks whether a dispatch happened while the monitor already suspected the shop. This distinction matters when an early batch on a day causes the surprise and a later batch on that same day detects it.

The CSV columns are day, date, closure-window marker, target ID, random-policy pieces, our-policy pieces, our pieces while already suspect, counts of new/cleared flags, and suspect state at the end of the day. There are 86 daily rows from day 0 through 85. Days without observations carry the last known flag state; the program is not polling a real workshop every day.

`main`, line 110, handles command-line settings and output. `--output` saves CSV; otherwise CSV goes to standard output. Diagnostic explanations go to standard error so they do not corrupt redirected CSV. `--events` saves the detailed JSON event evidence. `--seed` and `--objective` allow other experiments. There is no need to add `--shock` to this script because it already enables the shock.

The saved default evidence is:

| Event | Observed result |
|---|---|
| Hidden target | W2 SteadyHands |
| First detection | Day 33, February 3, while deciding ORD-047 |
| Expected queue | 5.3824 days |
| Observed queue | 13.9412 days |
| Unexpected jump | 8.5588 days, above the 3-day threshold |
| Our work on detection day | 500 pieces were sent earlier that day, before detection |
| Our work while W2 remained flagged | 0 pieces in this default trace |
| Random work on days 34-43 | 550 pieces: 400 on day 34 and 150 on day 38 |
| Our work on days 34-43 | 0 pieces |
| Recovery | Day 44, February 14; observed queue 2.9412 versus expected 0, now within threshold |
| Dispatch after recovery | W2 receives work again; it is not permanently blacklisted |

We can honestly say the monitor detects an observable disruption and the robustness setting avoids the suspect in this trace. We cannot say it predicted the closure before evidence appeared, always detects every closure, or always produces the fastest overall schedule.

# 13. Tests and measured performance

`tests/conftest.py` locates the project root and puts `src/` and `harness/` on Python's import search path. That lets tests say `from tools import ...` and `from simulate import ...`. It is test plumbing, not allocation logic.

A test supplies a situation and asserts an expected result. A fixture prepares reusable inputs; parameterisation runs the same check for multiple examples. `pytest.approx` compares decimal calculations with a small numerical tolerance. `pytest.raises` checks that an expected error is raised. Tests using temporary paths write temporary example config/output files rather than rewriting the real dataset.

The current suite passed **89 test cases** during this review:

| Test file | Cases | What is checked |
|---|---|---|
| `test_tools.py` | 38 | Ten seeded random-batch eligibility comparisons with the official helper; OldMill; FreshStart cap and boundary; Little Loom TOPS refusal; reason precedence; hand ETA/cost/queue calculations; exclusions, sorting, JSON compatibility, no input mutation, and selected invalid inputs |
| `test_allocator.py` | 37 | Presets and exact score arithmetic; JSON/YAML loading and invalid configs; normal and shock simulations for all six presets; legal selections; JSON records; date/exclusion/awkward cases; state updates; deterministic replay; heuristic behaviour |
| `test_shock_monitor.py` | 14 | Expected queue ageing; strict threshold; never-visited shops; same-day observations; recovery; redispatch while suspect; invalid thresholds/backwards time; DecisionRecord evidence; trace command from another directory |

The six-preset, two-scenario integration checks alone exercise twelve 120-order simulation runs. A passing test suite is evidence that these checked behaviours work. It is not 89% accuracy, 89 successful chat conversations, proof of real-world reliability, or proof that we beat every policy.

## Fresh comparison from this review

All figures below use the supplied 120 orders and seed 5105, with a fresh allocator per run. Displayed figures are rounded; calculation used full precision. These are development/review measurements, not a claimed held-out final evaluation. This review did not implement a new permanent benchmark runner.

### Normal run

| Policy | Mean days | P90 days | Late batches / 120 | Late % | Defect % | Cost | Max piece share % |
|---|---|---|---|---|---|---|---|
| random | 21.52 | 45.62 | 41 | 34.17 | 4.17 | 111,780 | 22.62 |
| greedy_biggest | 49.02 | 80.62 | 101 | 84.17 | 4.17 | 108,430 | 70.16 |
| cheapest | 105.13 | 192.87 | 112 | 93.33 | 7.50 | 85,960 | 70.16 |
| earliest_finish | 8.05 | 13.69 | 2 | 1.67 | 5.00 | 115,640 | 25.56 |
| lateness | 8.05 | 13.69 | 2 | 1.67 | 5.00 | 115,640 | 25.56 |
| defect | 162.51 | 282.15 | 114 | 95.00 | 1.67 | 129,920 | 70.16 |
| fairness | 19.20 | 54.57 | 28 | 23.33 | 5.00 | 116,045 | 17.49 |
| robustness | 19.20 | 54.57 | 28 | 23.33 | 5.00 | 116,045 | 17.49 |
| hybrid_lateness_fairness | 12.20 | 25.44 | 6 | 5.00 | 5.00 | 113,385 | 21.02 |
| hybrid_lateness_defect | 20.42 | 33.23 | 19 | 15.83 | 4.17 | 109,360 | 41.28 |

### Shock run

| Policy | Mean days | P90 days | Late batches / 120 | Late % | Defect % | Cost | Max piece share % |
|---|---|---|---|---|---|---|---|
| random | 23.36 | 48.35 | 47 | 39.17 | 4.17 | 111,780 | 22.62 |
| greedy_biggest | 51.04 | 81.25 | 104 | 86.67 | 4.17 | 108,430 | 70.16 |
| cheapest | 103.88 | 190.04 | 110 | 91.67 | 5.00 | 85,960 | 70.16 |
| earliest_finish | 8.38 | 15.38 | 2 | 1.67 | 5.00 | 115,805 | 26.15 |
| lateness | 8.38 | 15.38 | 2 | 1.67 | 5.00 | 115,805 | 26.15 |
| defect | 165.00 | 285.12 | 114 | 95.00 | 1.67 | 129,920 | 70.16 |
| fairness | 20.17 | 56.71 | 32 | 26.67 | 4.17 | 116,045 | 17.49 |
| robustness | 18.61 | 47.71 | 31 | 25.83 | 3.33 | 115,750 | 17.33 |
| hybrid_lateness_fairness | 13.77 | 31.43 | 19 | 15.83 | 4.17 | 113,195 | 20.05 |
| hybrid_lateness_defect | 19.72 | 34.00 | 26 | 21.67 | 2.50 | 109,180 | 40.53 |

For the normal run, our lateness setting finishes in 8.05 days on average, compared with random's 21.52. It has 2 late batches out of 120, compared with random's 41. However, **earliest_finish gives exactly the same outcomes as the lateness allocator on both these normal and shock runs**. The correct claim is that the richer implementation matches that strong short baseline here; it does not beat it.

The cheapest policy wins on cost. The pure defect setting has a lower realised batch defect rate but severe lateness: it repeatedly favours limited high-quality capacity and builds a queue. Fairness distributes pieces

more evenly, but smaller workshops can take longer. The hybrids expose trade-offs rather than producing a universally best answer.

The shock robustness setting averages 18.61 days, whereas earliest finish averages 8.38. Successfully avoiding a flagged workshop is a narrower success than winning overall turnaround. We should show both findings.

The simulator's printed report rounds percentages to whole percentages, so 2/120 late batches may display as 2%; the more precise rate is about 1.67%. Cost is not labelled with an invented currency. A p90 of 13.69 days describes the slower end of the batch completion distribution under this harness's percentile definition, not a guaranteed maximum.

One subtle comparison limit: enabling shock consumes an extra random draw to choose the workshop, changing the subsequent defect draws relative to a no-shock run. Some normal-versus-shock differences therefore reflect rework randomness as well as the closure. Multiple seeds and carefully described comparisons are useful, but extra seeds alone do not make the same development orders into unseen test data.

## 14. What to implement next, and why

1. **Agree the business objective and evaluation contract.** Choose and justify the team's primary objective for Sprint 1. Keep the other-objective and hybrid requirements. Decide how to balance lateness, quality, fairness, disruption, and cost, and identify which conditions are hard constraints. Do not change weights only to make one chart look good.
2. **Label the 30 chat examples before building the language layer.** Record the expected behaviour, known facts, missing facts, permissible workshop decisions, and claims an explanation may make. Include past deadlines, invalid workshop suggestions, unknown orders, and exclusions. Record which examples are used for development and reserve additional unseen examples for evaluation.
3. **Build structured intake and validation.** Resolve an order ID, category, positive integer quantity, dispatch/current date, due date, exclusions, and objective preference. Preserve who imposed each constraint. Ask about missing or conflicting facts. Never substitute customer selling price with workshop cost or invent past allocation history.
4. **Add the language model as the conversational coordinator.** It should interpret a message, ask a follow-up, invoke the calculation tools, and explain returned evidence. The arithmetic stays in Python. A single well-tooled dispatcher is sufficient; the assignment does not require multiple agents, fine-tuning a model, or a complex framework. These small CSVs can be looked up directly without first building a vector database.
5. **Separate preview, confirmation, and persistent recording.** A dispatcher should be able to compare alternatives without changing history on every screen refresh. Add protection against counting or committing the same request twice. Save full decision records, objective weights, original messages, source constraints, queue snapshots, and real timestamps. The current program makes simulated decisions; it does not contact actual subcontractors.
6. **Build the Streamlit interface.** Show the request, unresolved details, exclusions, objective, legal alternatives, ETA/cost/quality, suspect evidence, recommendation, and explicit awkward-case options. Handle session state and reset deliberately. The current dependencies prepare for this but there is no runnable Streamlit app yet.
7. **Decide whether splitting belongs in the completed scope.** If included, actually compute feasible pieces per workshop, respect caps, recompute queues and final arrival, and keep quantities adding up to

the original batch. The existing harness accepts one workshop ID per batch; split evaluation needs a separately documented approach without silently modifying the shared benchmark. Merely suggesting a split must be labelled as a suggestion.

8. **Turn evaluation into repeatable evidence.** Build a reusable runner/report covering the primary setting, another objective, a hybrid, all three supplied baselines, and earliest finish, under normal and shock conditions. Include other seeds, declared held-out inputs, weight/threshold sensitivity, and an otherwise comparable monitor-off experiment. Evaluate all 30 messages by behaviour, separate decision correctness from explanation faithfulness, and analyse at least ten actual individual failures. Once a model is added, measure latency and model cost as well.
9. **Close documented limitations and prepare handover.** Address the date-rounding adapter issue, user-input validation, unknown exclusions, persistence, and duplicate requests. Expand setup instructions to run the real app from a clean checkout. Prepare diagrams, demonstration, documentation, contributions, and evidence-based strategic recommendations.

## 15. Presentation preparation and deadlines

The supplied v4 schedule gives these dates; this guide is not a check for announcements made outside the supplied documents.

| Milestone | Supplied deadline and format |
|---|---|
| Sprint 1 | Friday 16 October 2026; slides by 3 PM, presentation at 7 PM; five-minute live presentation and two-minute Q&A |
| Sprint 2 | Friday 30 October 2026; video/slides by 3 PM, session at 7 PM; five-minute recorded presentation and two-minute live Q&A |
| Sprint 3 | Friday 13 November 2026; video/slides by 3 PM, session at 7 PM; five-minute recorded presentation and two-minute live Q&A |
| Final submission | Sunday 15 November 2026, 11:59 PM; `GroupNN.zip` |

Every team member should attend and be ready for questions. Not everyone must speak in the video. Use a two-digit group number and the exact specified capitalisation: `Presentation_GroupNN_Sprint1`, `Presentation_GroupNN_Sprint2`, and `Presentation_GroupNN` for the final presentation, with the appropriate slide/video extensions.

Sprint 1 should cover the problem and proposed solution with architecture, business value, optional early progress, risk, assumptions, and the Sprint 2 roadmap with success criteria. Declare your primary objective and explain the choice. The example PDF is a fictional email-assistant presentation template: use its clarity and architecture style, but do not present its features, market claims, or progress as ours.

For later sprints, show implemented versus planned features with an owner and status, actual validation, and a working demo. The first slide should include group details, members' student IDs, a public GitHub link, and a working demo link. A source repository alone is not a user-facing demo.

The final ZIP contains seven deliverables: `Readme.pdf`, `Slide.pdf` or `Slide.pptx`, `Video.mp4`, `Documentation.pdf` (maximum 15 pages), `Evaluation.pdf`, `Teams.pdf`, and `Strategic_Recommendations.pdf`. The final `Video.mp4` is a demo video, distinct from the Sprint 3 presentation video. Strategic recommendations require three specific industry opportunities backed by real market research, not generic claims. This learning guide is not a substitute for those final artifacts.

### A truthful explanation you can say aloud

*Our project helps a garment-factory dispatcher choose an outside workshop for each batch. We check product capability, suspension status, and batch limits, then calculate waiting time, production time, transport, cost, and defect risk. A configurable scoring rule ranks the legal choices, and we create a record explaining the decision. We also compare expected and observed queues to detect a possible disruption. The current implementation is a deterministic decision engine; we still need to add the conversational AI and dispatcher interface. In our seed-5105 experiments, the lateness setting beats the three supplied baselines on turnaround and lateness, but it matches the simple earliest-finish heuristic. We will report that honestly and evaluate the extra value of the conversational workflow and auditability.*

| Likely question | Answer you should understand |
|---|---|
| Where is AI used now? | ChatGPT helped us build the project, but the current runtime does not call an LLM. The conversational component remains to be implemented. |
| Why keep calculations in tools? | Explicit arithmetic is reproducible and inspectable; the language model can explain those results without inventing numbers. |
| Why not always choose the biggest workshop? | It may already have a long queue or longer transport time, and repeated allocations overload it. |
| Why not always choose the cheapest? | Lowest unit cost can create very long delays. |
| Is the ETA guaranteed? | No. It is nominal and can change with rework, hidden closures, stale inputs, and the documented date-rounding edge case. |
| Is a flagged workshop known to be closed? | No. The evidence is unexpected queue growth; there may be another cause. |
| Does the allocator learn? | It remembers counts and predictions, but it does not train a model or learn its scoring weights. |
| Why have a simple baseline? | To check whether added complexity improves outcomes and to identify where our contribution is workflow, constraints, or explanations instead. |
| What does 89 tests mean? | The checked programming behaviours passed; it is not a natural-language accuracy measure. |
| Is splitting implemented? | No. It is currently an explicitly labelled option for a later feasibility check. |
| What happens if the deadline is impossible? | The record marks that condition, chooses the least-late permissible shop, and lists options rather than claiming the date can be met. |
| Is there a database or actual workshop dispatch? | Not yet. Inputs are local files and the decisions are simulated; persistent logging and external workflow integration are separate work. |

## 16. Commands to explore the implementation yourself

In Terminal, move to this project and activate its existing environment:

```
cd "/Users/prateekbansal/Apurwa/Study/NUS_MS/DSS5105-Data_Science_Project_in_Practice/CodePrac
  tice/Track2_dispatch"
source .venv/bin/activate
```

For a fresh checkout without an environment, create it first with `python3 -m venv .venv`, allow that command to finish, activate it, then run `python -m pip install -r requirements.txt`. The earlier

`KeyboardInterrupt` meant environment creation was interrupted with Ctrl-C; an activation file may not yet have existed. Java startup output was unrelated to this Python project.

Run the supplied baselines and the tests:

```
python harness/simulate.py
python harness/simulate.py --shock
python -m pytest -q
```

Inspect the default shock trace without overwriting the saved files:

```
python eval/shock_trace.py
```

The documented command below regenerates the two saved evaluation files:

```
python eval/shock_trace.py --output eval/shock_trace.csv --events eval/shock_events.json
```

To compare the lateness allocator with the strong heuristic in both scenarios, run this from the project root. The lines between `PY` markers are Python rather than shell commands:

```
PYTHONPATH=src:harness python - <<'PY'
from allocator import Allocator
from objectives import PRESETS
from heuristic import earliest_finish
from simulate import Simulator

for shock in (False, True):
    print(f"\nShock enabled: {shock}")
    simulator = Simulator(shock=shock, seed=5105)
    simulator.run(earliest_finish, name="earliest_finish")
    simulator.run(Allocator(PRESETS["lateness"]), name="our_lateness")
    simulator.report()
PY
```

Running `python src/tools.py` by itself usually prints nothing: that file defines reusable tools but has no command-line demonstration. Similarly, importing `Allocator` only makes it available; you must construct it and call it or supply it to the simulator.

# 17. Where to edit when a requirement changes

| Desired change | Start here | Also consider |
|---|---|---|
| Choose different business preferences | `src/objectives.py` or a JSON/YAML config passed to `load_config` | Document the reason and rerun evaluation |
| Change ranking/fallback/constraint behaviour | `src/allocator.py` | Update decision tests and explanation expectations |
| Add a new estimate | `src/tools.py` | Keep arithmetic auditable, document assumptions in `docs/tool_spec.md`, and test against a hand calculation |
| Change anomaly sensitivity | `ShockMonitor` threshold and allocator construction | Test false alarms, missed delays, recovery, and outcome trade-offs |
| Add chat understanding | A new intake/agent module | Validate structured fields and preserve original evidence |
| Add a screen | A new Streamlit entry point | Avoid calling committing `decide()` repeatedly during previews |
| Save all decisions | A logging component and caller integration | Save original constraints, weights, timestamps, and snapshots; respect ignored local record files |
| Add evaluation charts/reports | `eval/` | Retain comparable baselines and disclose development/held-out use |
| Change workshop data for an extension | A clearly documented separate scenario/data copy | Preserve the supplied benchmark and distinguish extension results |

Do not change the shared simulator just to make our policy look better. Read the relevant existing tests before editing, make the smallest clear change, run the tests that exercise it, and compare the behaviour you intended with the actual outputs.

For study, begin with the business story and workshop table, reproduce the 340-piece hand calculation, follow the six tools, then read scoring, allocator state, and the monitor. Finish by running tests and explaining why the simple baseline sometimes wins. Once you can explain those relationships in your own words, the Python syntax becomes much easier to follow.